Chapter 5
Lists (cont) - Linked List

Data Structures and Algorithms

Dr. Nguyen Ho Man Rang
Faculty of Computer Science and Engineering
University of Technology, VNU-HCM

Outcomes

- **L.O.2.1** - Depict the following concepts: (a) array list and linked list, including single link and double links, and multiple links.
- **L.O.2.2** - Describe storage structures by using pseudocode for: (a) array list and linked list, including single link and double links, and multiple links.
- **L.O.2.3** - List necessary methods supplied for list and describe them using pseudocode.
- **L.O.2.4** - Implement list using C/C++.

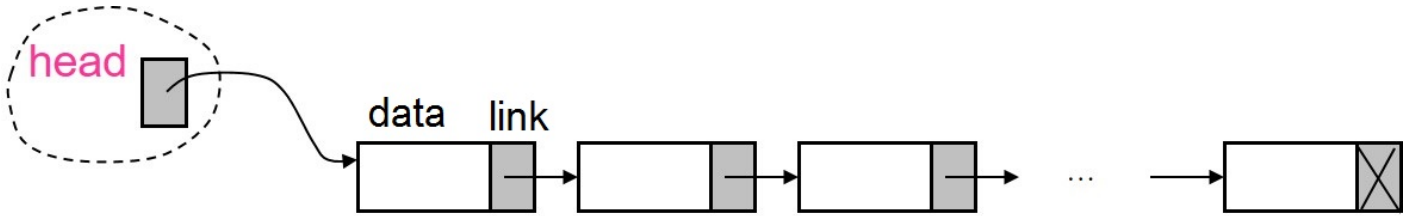
Outcomes

- **L.O.2.5** - Use list for problems in real-life, and choose an appropriate implementation type (array vs. link).
- **L.O.2.6** - Analyze the complexity and develop experiment (program) to evaluate the efficiency of methods supplied for list.
- **L.O.8.4** - Develop recursive implementations for methods supplied for the following structures: list.
- **L.O.1.2** - Analyze algorithms and use Big-O notation to characterize the computational complexity of algorithms composed by using the following control structures: sequence, branching, and iteration (not recursion).

Contents

Singly linked list

Linked List



Hình: Singly Linked List

```
list // Linked Implementation of List
  head <pointer>
  count <integer> // number of elements (optional)
end list
```

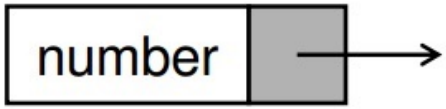

Dr. Nguyen Ho Man Rang

Nodes

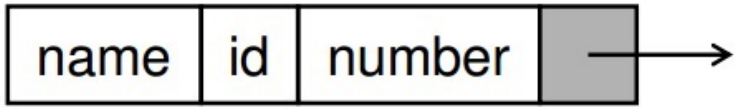
The elements in a linked list are called nodes.
A node in a linked list is a structure that has at least two fields:

- the data,
- the address of the next node.

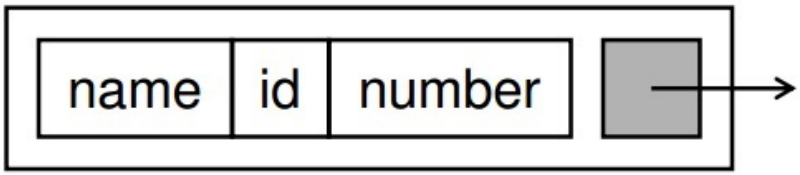
A node with one data field



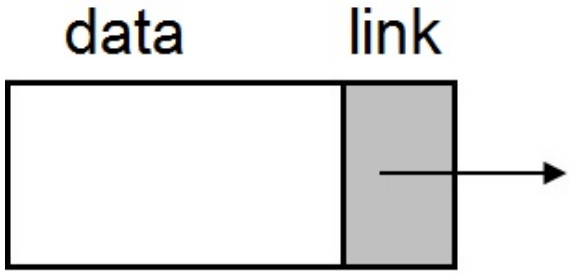
A node with three data fields



A node with one structured data field



Nodes

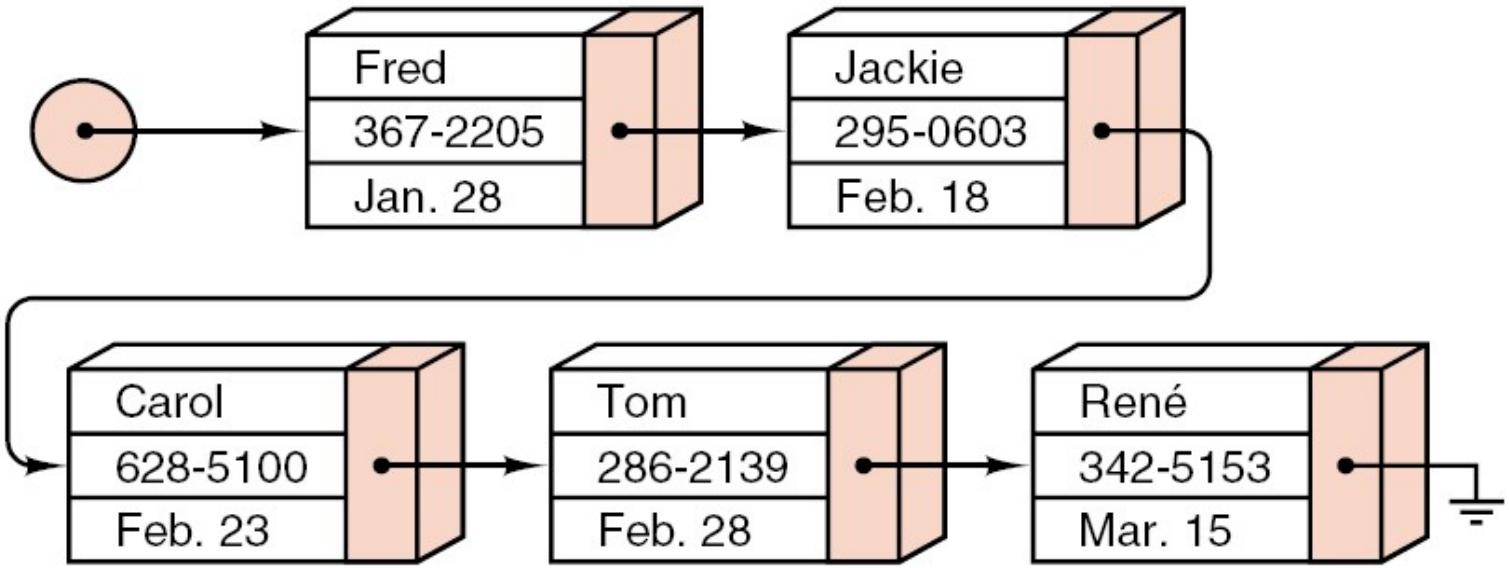


Hình: Linked list node structure

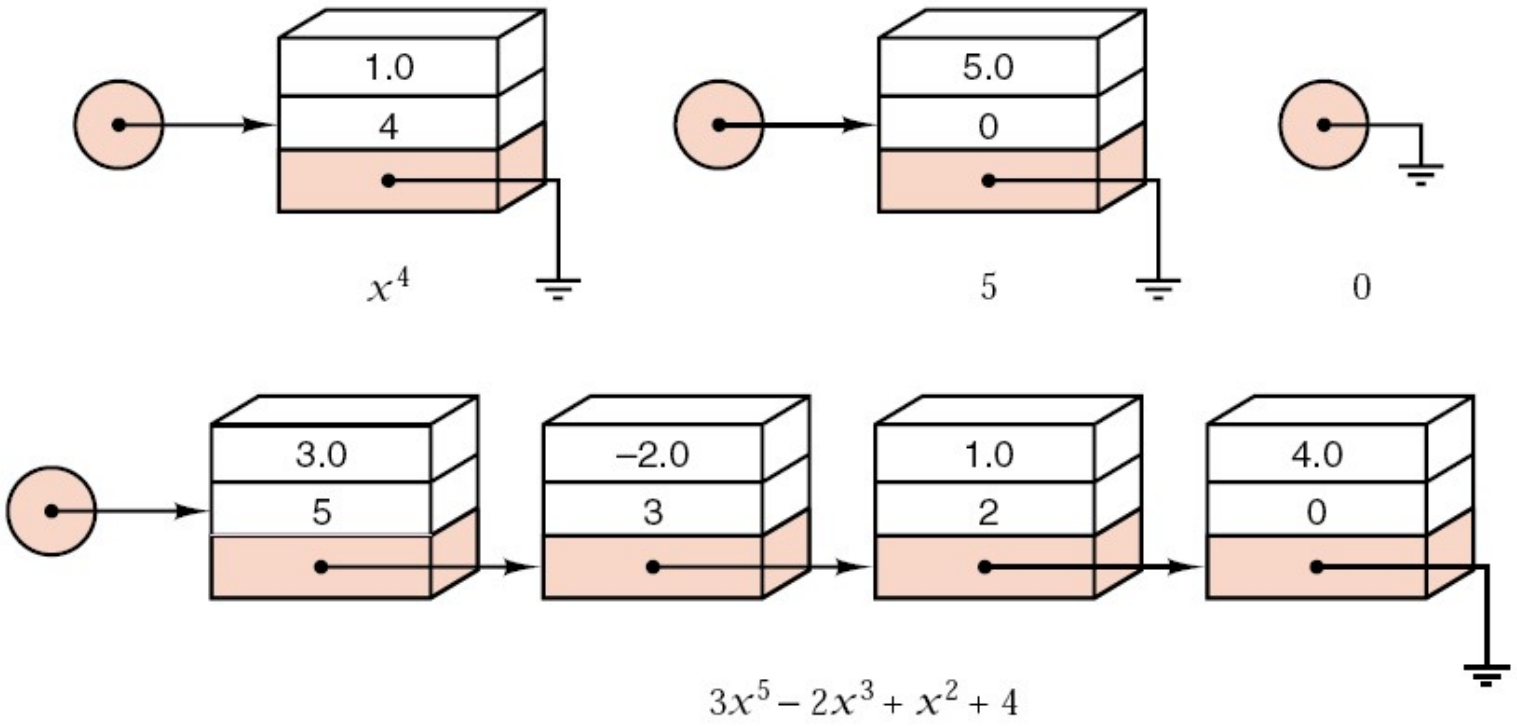
```
node
   data <dataType>
   link <pointer>
end node
```

```
// General dataType:
dataType
    key <keyType>
    field1 <...>
    field2 <...>
    ...
    fieldn <...>
end dataType
```


Example


Fred
367-2205
Jan. 28
Jackie
295-0603
Feb. 18
Carol
628-5100
Feb. 23
Tom
286-2139
Feb. 28
René
342-5153
Mar. 15

Example



Hình: List representing polynomial

Dr. Nguyen Ho Man Rang

Implementation in C++

Example

```
node
  data <dataType>
  link <pointer>
end node
```

```
struct Node {
    int data;
    Node *link;
};
```


Implementation in C++

Example

```
struct Node {
    int data;
    Node *link;
};
```

```
struct Node {
    float data;
    Node *link;
};
```

```
template <class ItemType>
struct Node {
    ItemType data;
    Node<ItemType> *link;
};
```

Node implementation in C++

```
class Node {
  public:
    int data;
    Node *link;
    Node(){
      this->link = NULL;
    }

    Node(int data){
      this->data = data;
      this->link = NULL;
    }
};
```


Dr. Nguyen Ho Man Rang

Linked list implementation in C++

```
class LinkedList {
  Node *head;
  int count;
public:
  LinkedList();
  ~LinkedList();
};
```

```
list
  head <pointer>
  count <integer>
end list
```

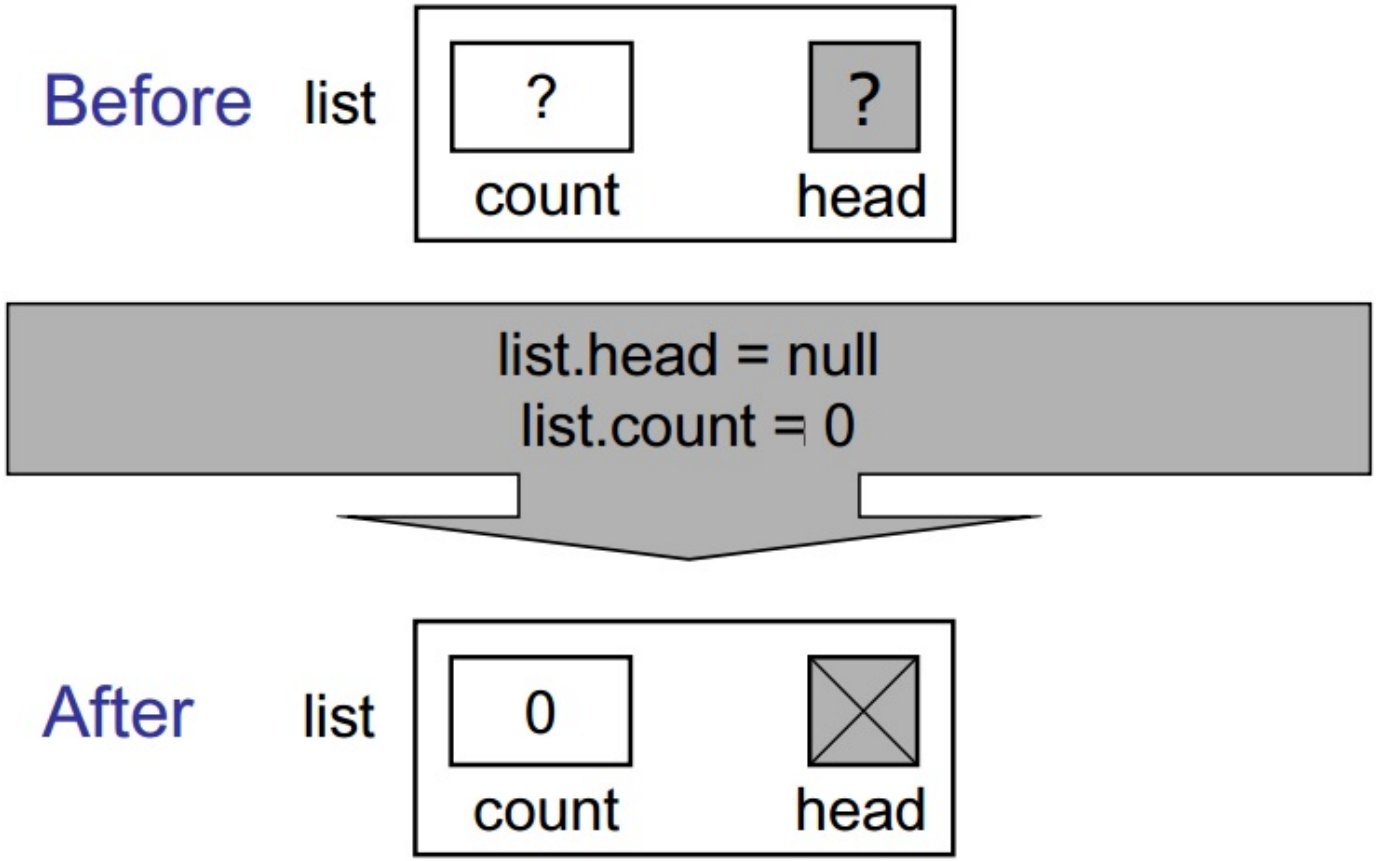

Dr. Nguyen Ho
Man Rang

Linked list operations

- Create an empty linked list
- Insert a node into a linked list
- Delete a node from a linked list
- Traverse a linked list
- Destroy a linked list

Dr. Nguyen Ho Man Rang

Create an empty linked list



Create an empty linked list

Algorithm createList(ref list <metadata>)
Initializes metadata for a linked list
Pre: `list` is a metadata structure passed by reference
Post: metadata initialized
list.head = null
list.count= 0
return
End createList

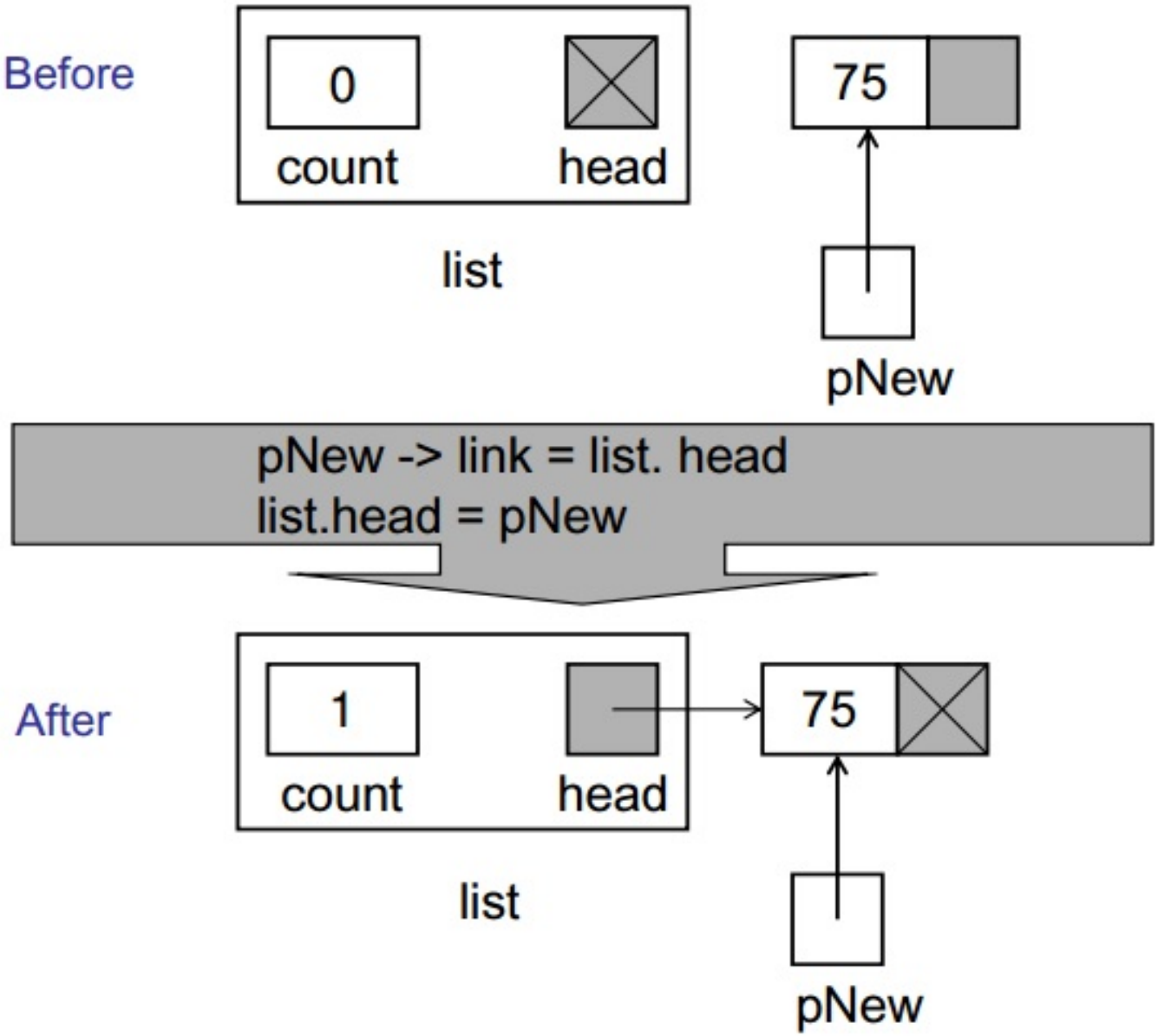
Create an empty linked list

```
class LinkedList {
  Node *head;
  int count;

  public:
    LinkedList();
    ~LinkedList();
};


LinkedList::LinkedList(){
  this->head = NULL;
  this->count = 0;
}
```


Insert a node into a linked list

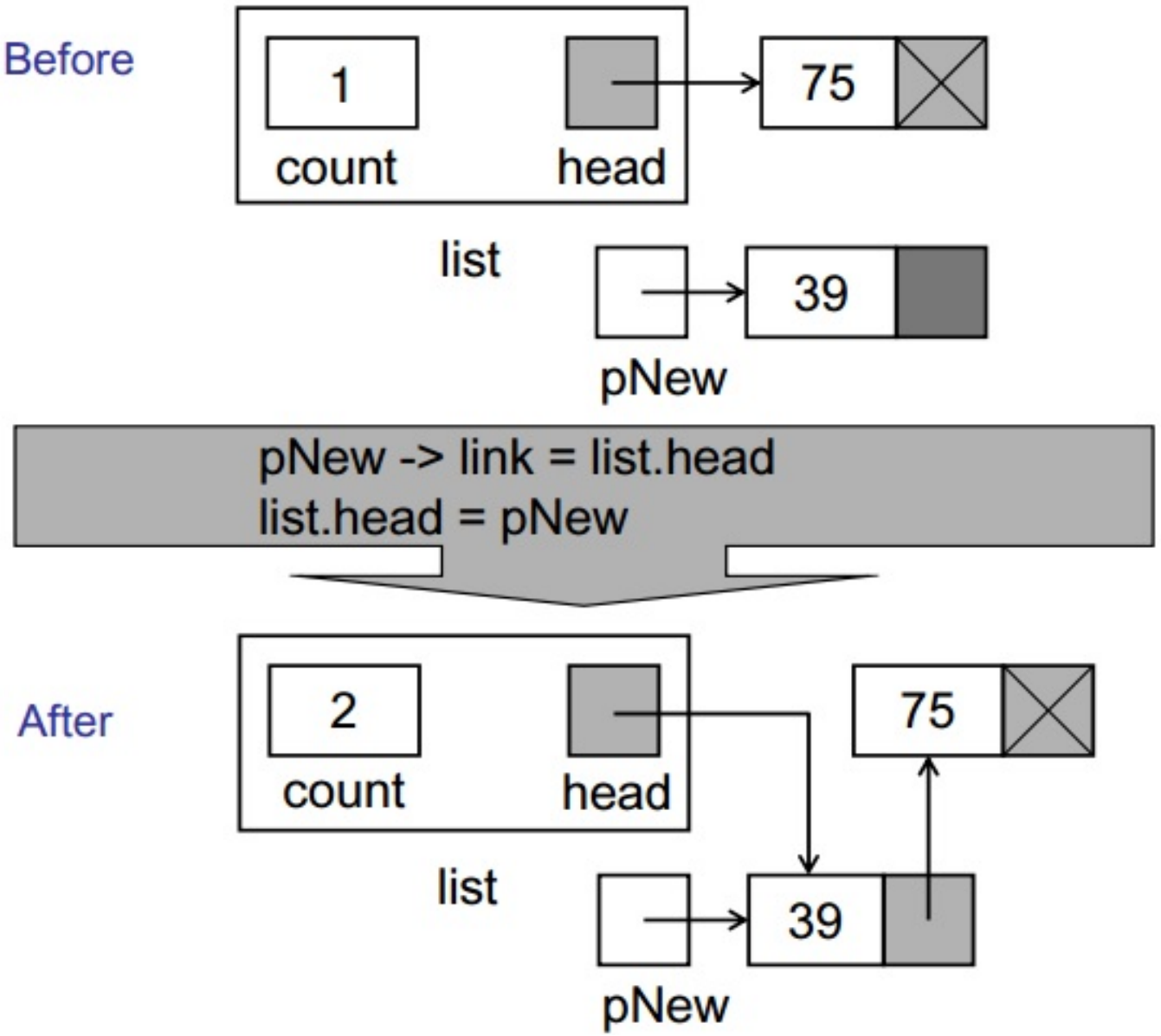
1. Allocate memory for the new node and set up data.
2. Locate the pointer p in the list, which will point to the new node:
   - If the new node becomes the first element in the List: `p is list.head`.
   - Otherwise: `p is pPre->link`, where `pPre` points to the predecessor of the new node.
3. Point the new node to its successor.
4. Point the pointer p to the new node.

Insert into an empty linked list



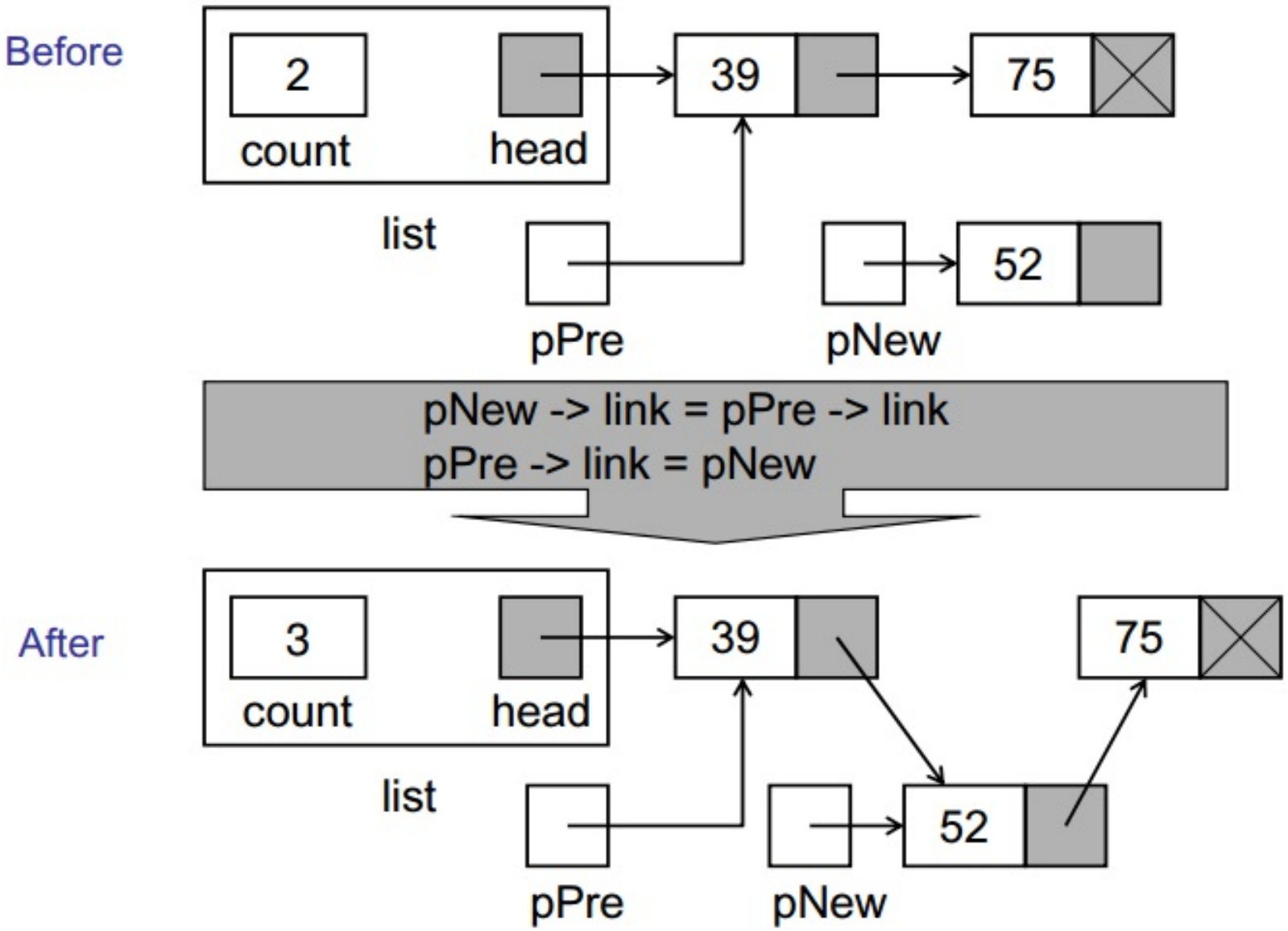
Dr. Nguyen Ho Man Rang

Insert at the beginning

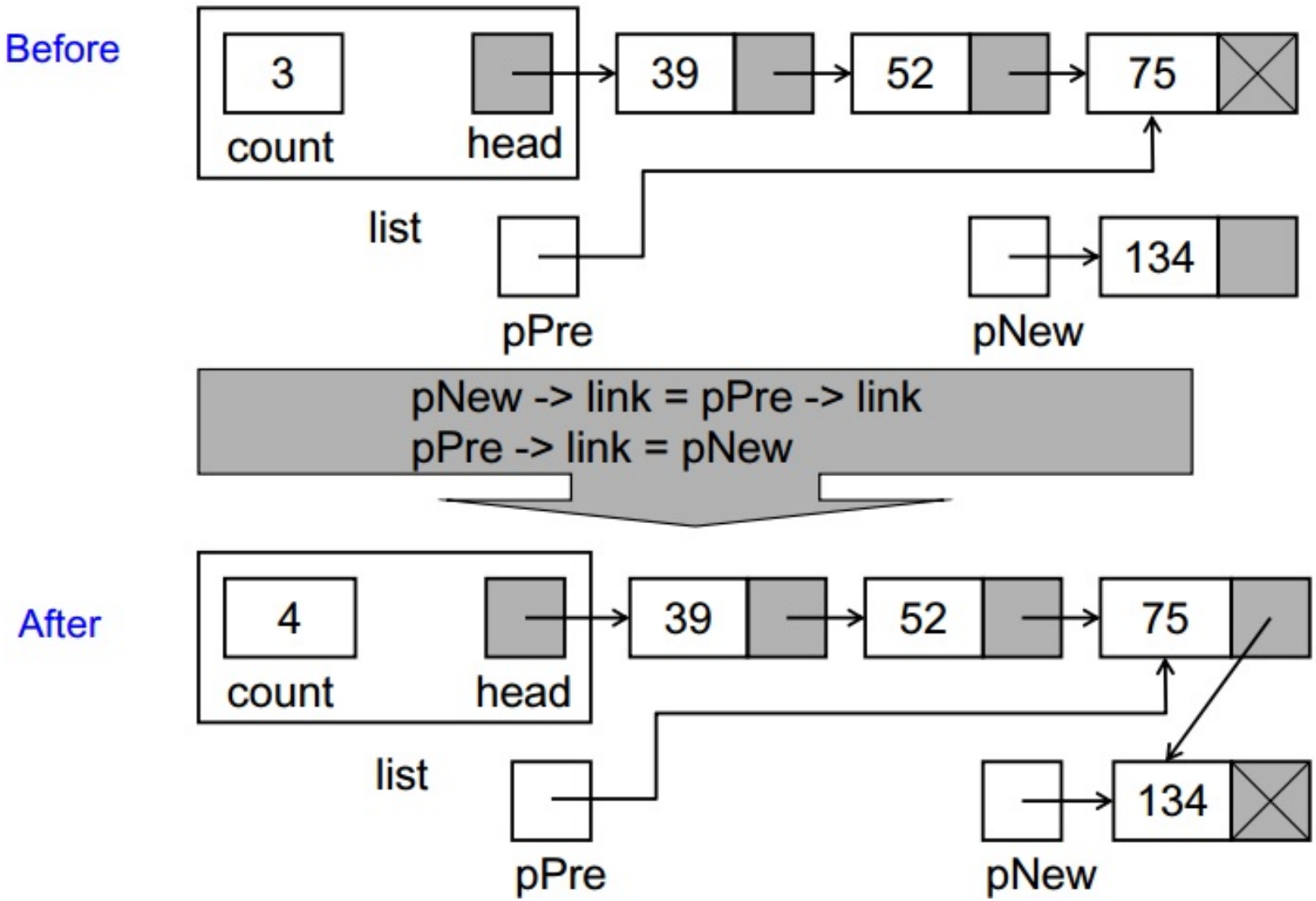


Dr. Nguyen Ho Man Rang

Insert in the middle



Insert at the end



Insert a node into a linked list

- Insertion is successful when allocation memory for the new node is successful.
- There is no difference between insertion at the beginning of the list and insertion into an empty list.

```
pNew->link = list.head
list.head = pNew
```

- There is no difference between insertion in the middle and insertion at the end of the list.

```
pNew->link = pPre->link
pPre->link = pNew
```


Insert a node into a linked list

Algorithm insertNode(ref list <metadata>,
val pPre <node pointer>,
val dataIn <dataType>)
Inserts data into a new node in the linked list.

Pre: `list` is metadata structure to a valid list
`pPre` is pointer to data's logical predecessor
`dataIn` contains data to be inserted
Post: data have been inserted in sequence
Return true if successful, false if memory overflow

Insert a node into a linked list

```
allocate(pNew)
if memory overflow then
    return false
end
pNew -> data = dataIn
if pPre = null then
    // Adding at the beginning or into empty list
    pNew -> link = list.head
    list.head = pNew
else
    // Adding in the middle or at the end
    pNew -> link = pPre -> link
    pPre -> link = pNew
end
list.count = list.count + 1
return true
End insertNode
```


Insert a node into a linked list

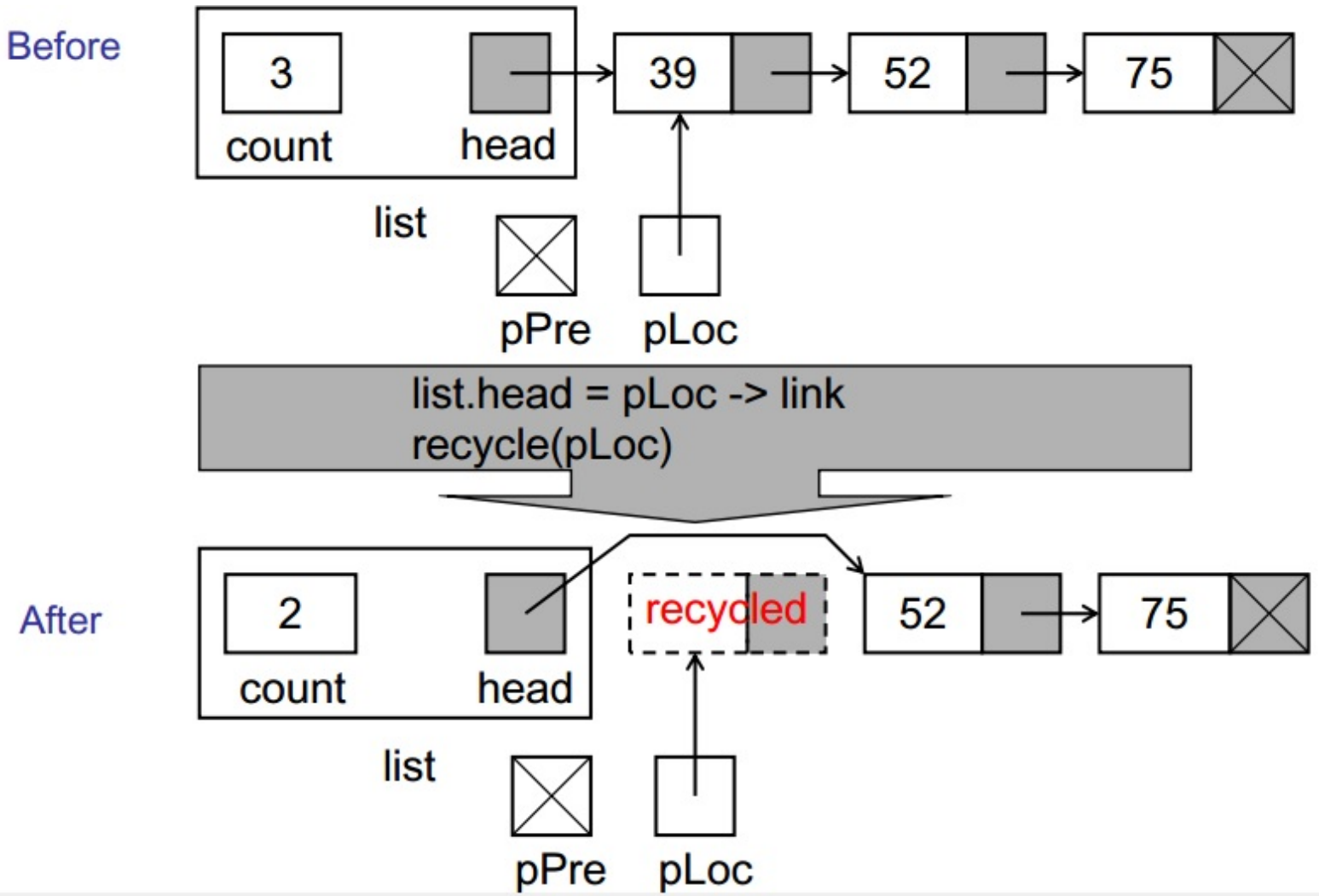
```
int LinkedList::InsertNode(Node*pPre,
        int value) {
  Node *pNew = new Node();
  if (pNew == NULL)
    return 0;
  pNew->data = value;
  if (pPre== NULL){
    pNew->link = this->head;
    this->head = pNew;
  } else {
    pNew->link = pPre->link;
    pPre->link = pNew;
  }
  this->count++;
  return 1;
}
```


Delete a node from a linked list

1. Locate the pointer p in the list which points to the node to be deleted (pLoc will hold the node to be deleted).
   - If that node is the first element in the List: `p is list.head`.
   - Otherwise: `p is pPre->link`, where `pPre` points to the predecessor of the node to be deleted.
2. `p` points to the successor of the node to be deleted.
3. Recycle the memory of the deleted node.

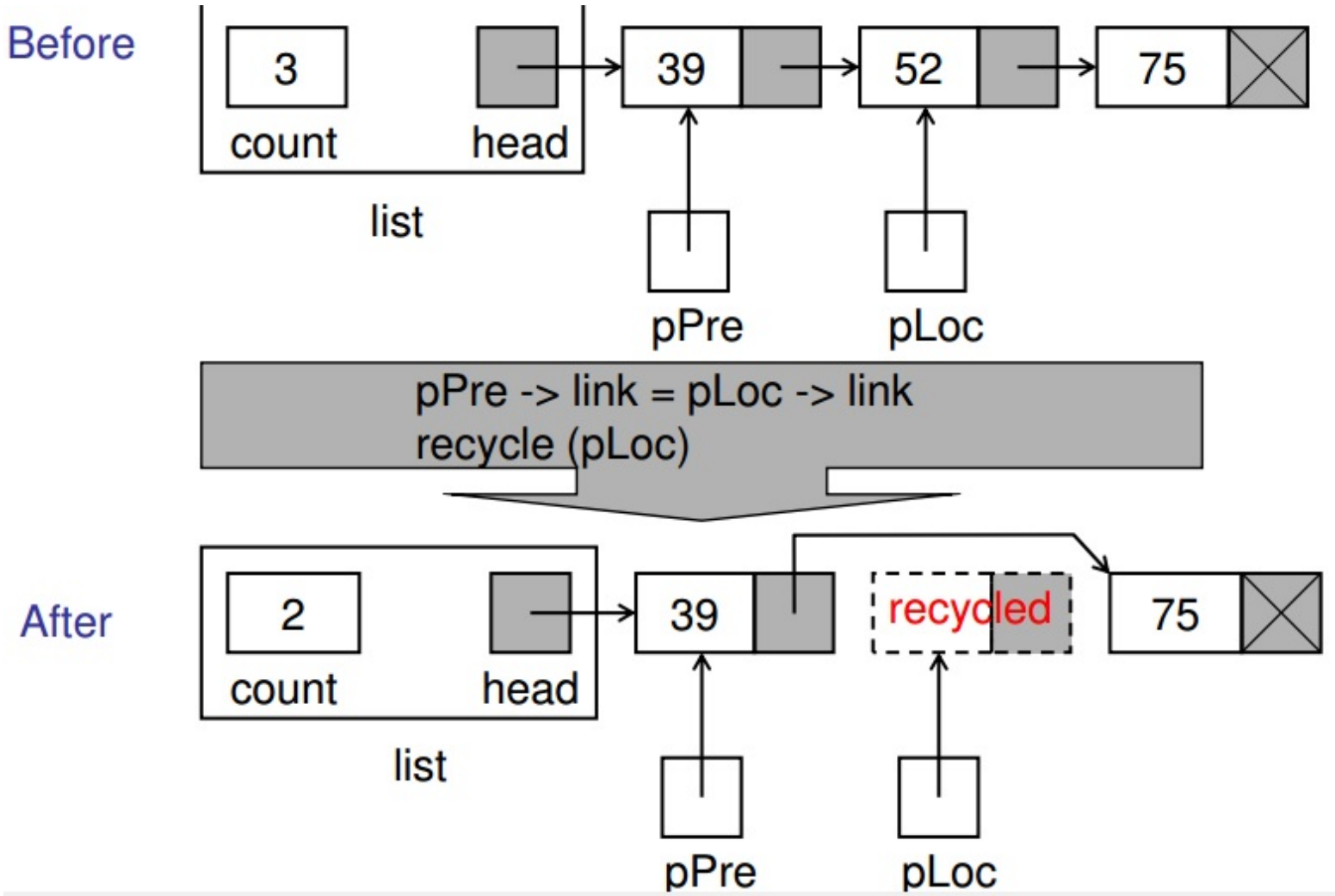
Dr. Nguyen Ho Man Rang

Delete first node



Dr. Nguyen Ho Man Rang

General deletion case



Delete a node from a linked list

- Removal is successful when the node to be deleted is found.
- There is no difference between deleting the node from the beginning of the list and deleting the only node in the list.

```
list.head = pLoc->link
recycle(pLoc)
```

- There is no difference between deleting a node from the middle and deleting a node from the end of the list.

```
pPre->link = pLoc->link
recycle(pLoc)
```


Delete a node from a linked list

Algorithm deleteNode(ref list <metadata>,
val pPre <node pointer>,
val pLoc <node pointer>,
ref dataOut <dataType>)
Deletes data from a linked list and returns it to calling module.

Pre: `list` is metadata structure to a valid list
`pPre` is a pointer to predecessor node
`pLoc` is a pointer to node to be deleted
`dataOut` is variable to receive deleted data
Post: data have been deleted and returned to caller

Dr. Nguyen Ho Man Rang

Delete a node from a linked list

```
dataOut = pLoc -> data
if pPre = null then
    // Delete first node
    list.head = pLoc -> link
else
    // Delete other nodes
    pPre -> link = pLoc -> link
end
list.count = list.count - 1
recycle (pLoc)
return
End deleteNode
```


Delete a node from a linked list

```
int LinkedList::DeleteNode(Node *pPre, Node *pLoc) {
  int result = pLoc->data;

  if (pPre== NULL){
    this->head = pLoc->link;
  } else {
    pPre->link = pLoc->link;
  }

  this->count--;
  delete pLoc;
  return result;
}
```

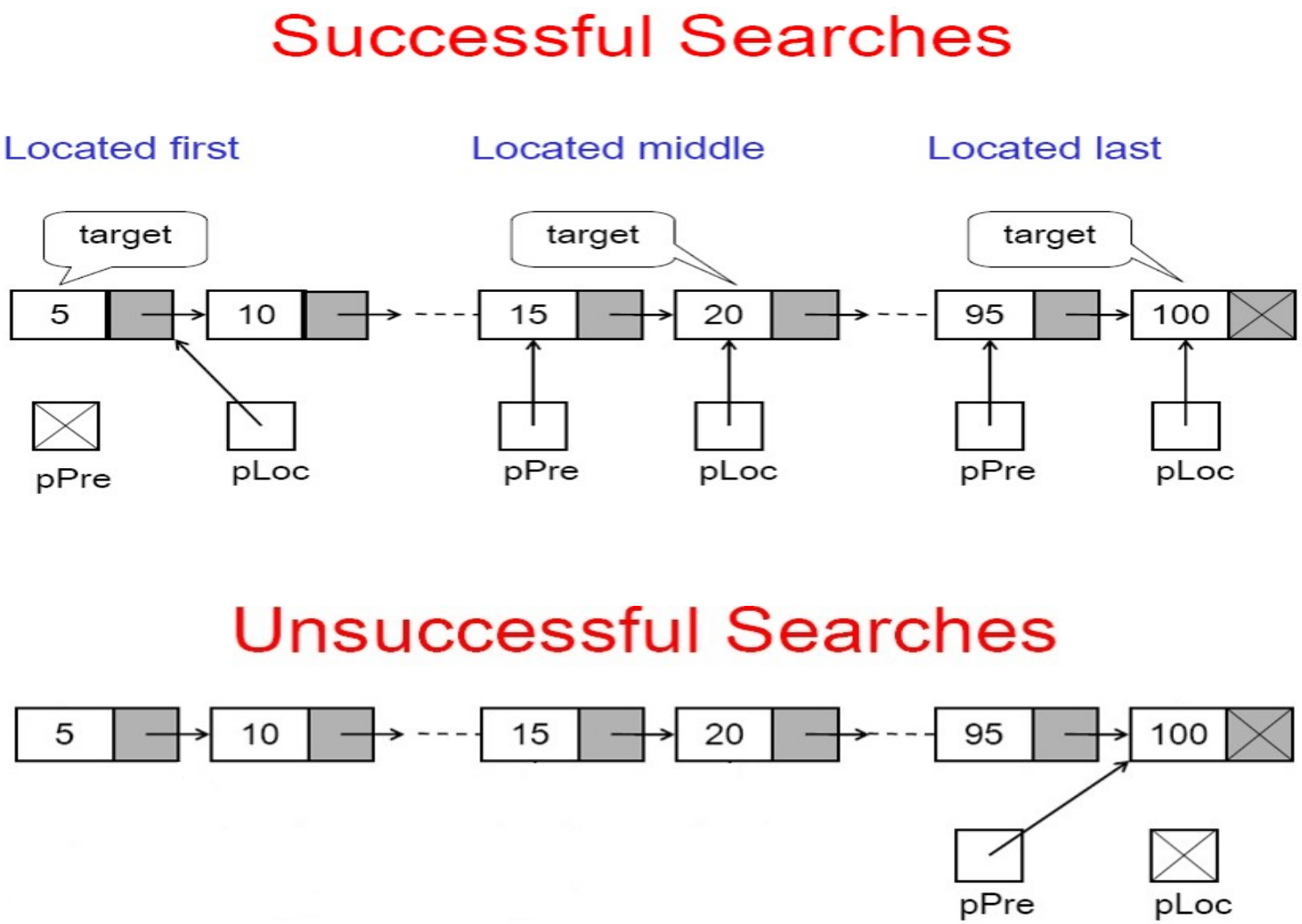

Searching in a linked list

- Sequence Search has to be used for the linked list.
- Function Search of List ADT:

```
<ErrorCode> Search (val target <dataType>,
        ref pPre <pointer>, ref pLoc <pointer>)
```

Searches a node and returns a pointer to it if found.

Searching in a linked list



Searching in a linked list

Algorithm Search(val target <dataType>,
ref pPre <node pointer>,
ref pLoc <node pointer>)
Searches a node in a singly linked list and return a pointer to it if found.

Pre: `target` is the value need to be found

Post: `pLoc` points to the first node which is equal target, or is NULL if not found.
`pPre` points to the predecessor of the first node which is equal target, or points to the last node if not found.

Return `found` or `notFound`

Searching in a linked list

```
pPre = NULL
pLoc = list.head
while (pLoc is not NULL) AND
(target != pLoc ->data) do
    pPre = pLoc
    pLoc = pLoc ->link
end
if pLoc is NULL then
    return notFound
else
    return found
end
End Search
```


Searching in a linked list

```
int LinkedList::Search(int value,
        Node* &pPre, Node* &pLoc){
  pPre = NULL;
  pLoc = this->head;
  while (pLoc != NULL && pLoc->data != value){
    pPre = pLoc;
    pLoc = pLoc->link;
  }
  return (pLoc != NULL);
  // found: 1; notfound: 0
}
```


Traverse a linked list

Traverse module controls the loop: calling a user-supplied algorithm to process data

Algorithm Traverse(ref <void> process (ref Data <DataType>))
Traverses the list, performing the given operation on each element.
Pre: `process` is user-supplied
Post: The action specified by `process` has been performed on every element in the list, beginning at the first element and doing each in turn.

```
pWalker = list.head
while pWalker not null do
    process(pWalker -> data)
    pWalker = pWalker -> link
end
End Traverse
```


Dr. Nguyen Ho Man Rang

Traverse a linked list

```
void LinkedList::Traverse() {
  Node *p = head;
  while (p != NULL){
    p->data++; // process data here!!!
    p = p->link;
  }
}

void LinkedList::Traverse2(int *&visit){
  Node *p = this->head;
  int i = 0;
  while (p != NULL && i < this->count){
    visit[i] = p->data;
    p = p->link;
    i++;
  }
}
```


Destroy a linked list

Algorithm destroyList (val list <metadata>)
Deletes all data in list.

Pre: `list` is metadata structure to a valid list
Post: all data deleted
while *list.head not null* **do**
    dltPtr = list.head
    list.head = this.head -> link
    recycle (dltPtr)
end
No data left in list. Reset metadata
list.count = 0
return
End destroyList

Dr. Nguyen Ho Man Rang

Destroy a linked list

```
void LinkedList::Clear(){
  Node *temp;
  while (this->head != NULL){
    temp = this->head;
    this->head = this->head->link;
    delete temp;
  }
  this->count = 0;
}


LinkedList::~LinkedList(){
  this->Clear();
}
```


Linked list implementation in C++

```
class LinkedList{
  Node* head;
  int count;
protected:
  int InsertNode(Node* pPre,int value);

  int DeleteNode(Node* pPre, Node* pLoc);

  int Search(int value, Node* &pPre, Node* &pLoc);

};
```


Dr. Nguyen Ho
Man Rang

Linked list implementation in C++

```
class LinkedList{

protected:
  // ...

public:
  LinkedList();
  ~LinkedList();

  void InsertFirst(int value);
  void InsertLast(int value);
  int InsertItem(int value, int position);

  void DeleteFirst();
  void DeleteLast();
  int DeleteItem(int postion);
  int GetItem(int position, int &dataOut);

  void Traverse();
  LinkedList* Clone();
  void Print2Console();
  void Clear();
};
```


Linked list implementation in C++

How to use Linked List data structure?

```
int main(int argc, char* argv[]) {
  LinkedList* myList = new LinkedList();
  myList->InsertFirst(15);
  myList->InsertFirst(10);
  myList->InsertFirst(5);
  myList->InsertItem(18,3);
  myList->InsertLast(25);
  myList->InsertItem(20,3);
  myList->DeleteItem(2);
  cout << "List 1:" << endl;
  myList->Print2Console();
```


Dr. Nguyen Ho
Man Rang

Linked list implementation in C++

How to use Linked List data structure?

```
    // ...

    int value;
    LinkedList* myList2 = myList->Clone();
    cout << "List 2:" << endl;
    myList2->Print2Console();
    myList2->GetItem(1, value);
    cout << "Value at position 1: " << value;

    delete myList;
    delete myList2;
    return 1;
}
```


Sample Solution: Insert

```
int LinkedList<List_ItemType>::InsertItem(
        int value, int position) {

  if (position < 0 || position > this->count)
    return 0;

  Node *newPtr, *pPre;
  newPtr = new Node();
  if (newPtr == NULL)
    return 0;

  newPtr->data = value;

  if (head == NULL) {
    head = newPtr;
    newPtr->link = NULL;
  } else if (position == 0) {
    newPtr->link = head;
    head = newPtr;
  }
```

Sample Solution: Insert

```
    else {
        // Find the position of pPre
        pPre = this->head;

        for (int i = 0; i < position-1; i++)
            pPre = pPre->link;

        // Insert new node
        newPtr->link = pPre->link;
        pPre->link = newPtr;
    }

    this->count++;
    return 1;
}
```


Dr. Nguyen Ho Man Rang

Sample Solution: Delete

```
int LinkedList::DeleteItem(int position){
  if (position < 0 || position > this->count)
    return 0;

  Node *dltPtr, *pPre;

  if (position == 0) {
    dltPtr = head;
    head = head->link;
  } else {
    pPre= this->head;
    for (int i = 0; i < position -1; i++)
      pPre = pPre->link;
    dltPtr = pPre->link;
    pPre->link = dltPtr->link;
  }

  delete dltPtr;
  this->count--;
  return 1;
}
```

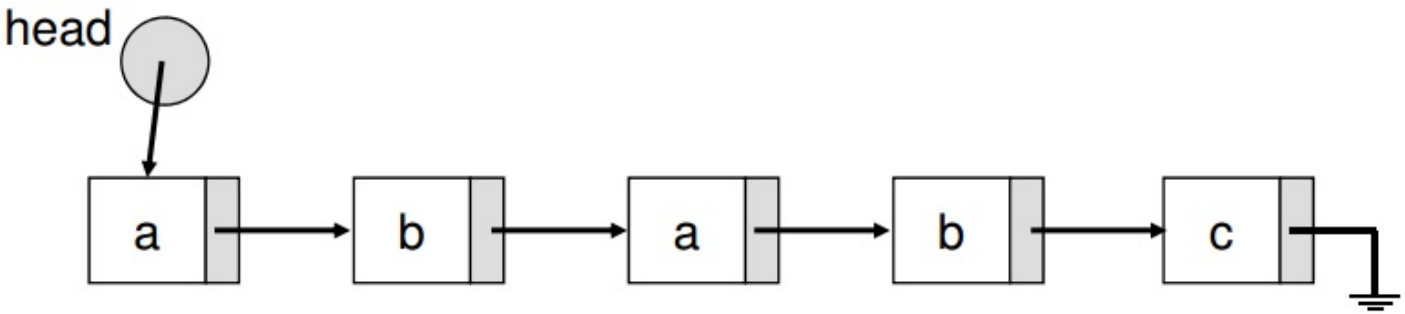
Sample Solution: Clone

```
LinkedList* LinkedList::Clone(){
  LinkedList* result = new LinkedList();

  Node* p = this->head;

  while (p != NULL) {
    result->InsertLast(p->data);
    p = p->link;
  }

  result->count = this->count;
  return result;
}
```

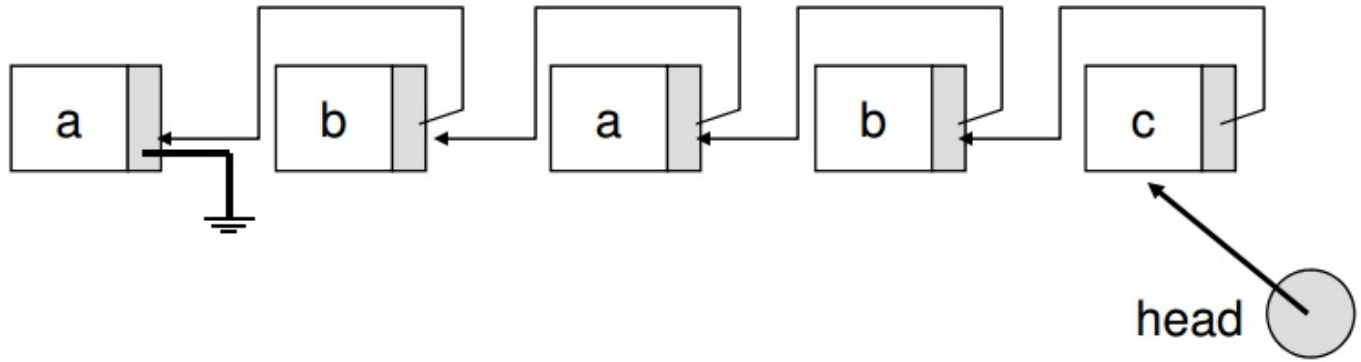

Reverse a linked list

Exercise

```
void LinkedList::Reverse(){
   // ...
}
```

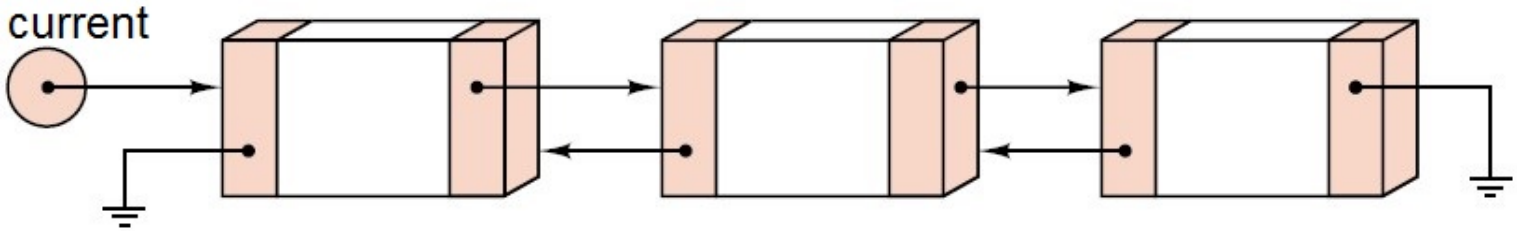


Result:



Other linked lists

Doubly Linked List

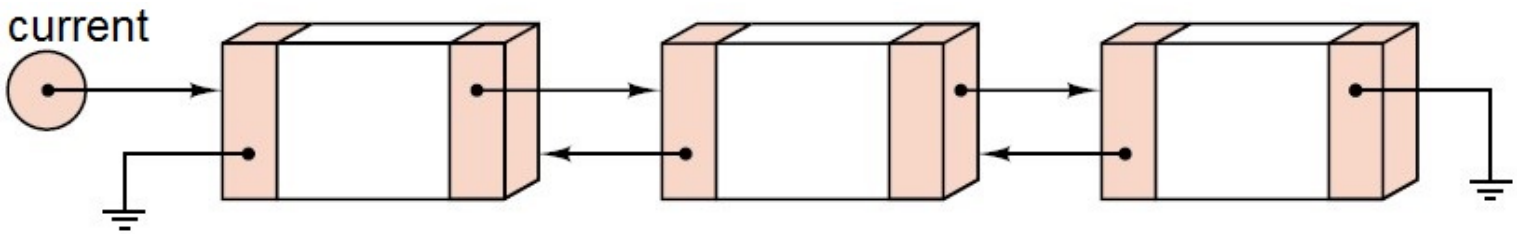


Hình: Doubly Linked List allows going forward and backward.

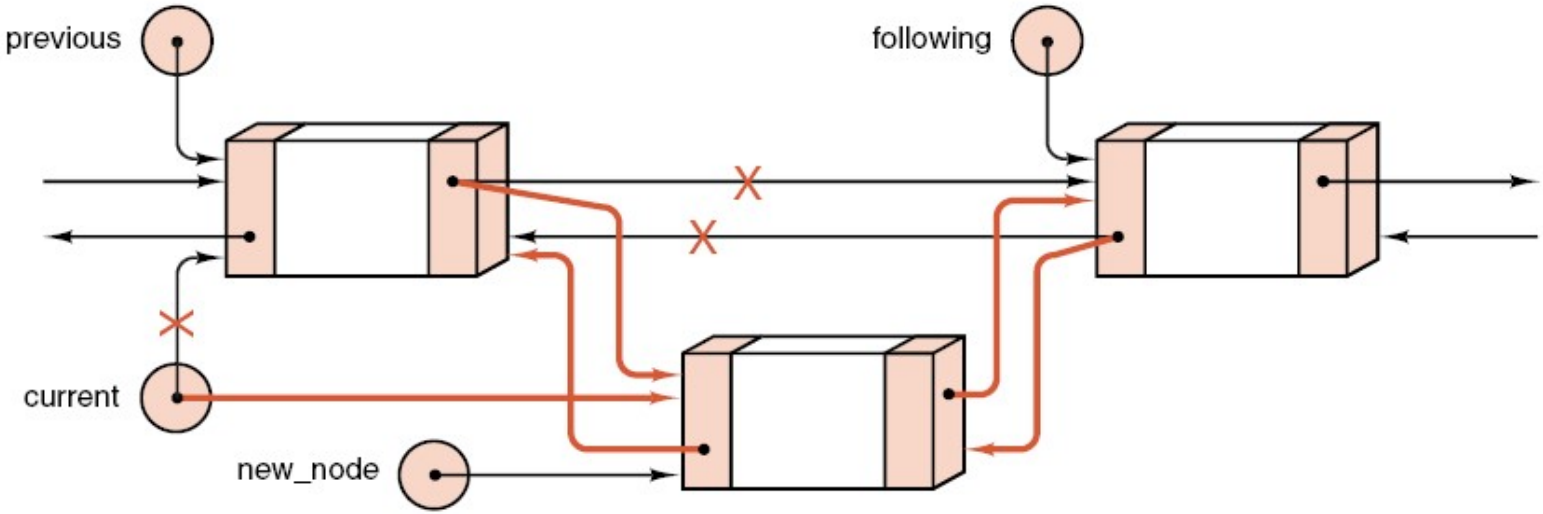
```
node
  data <dataType>
  next <pointer>
  previous <pointer>
end node
```

```
list
  current <pointer>
end list
```


Doubly Linked List

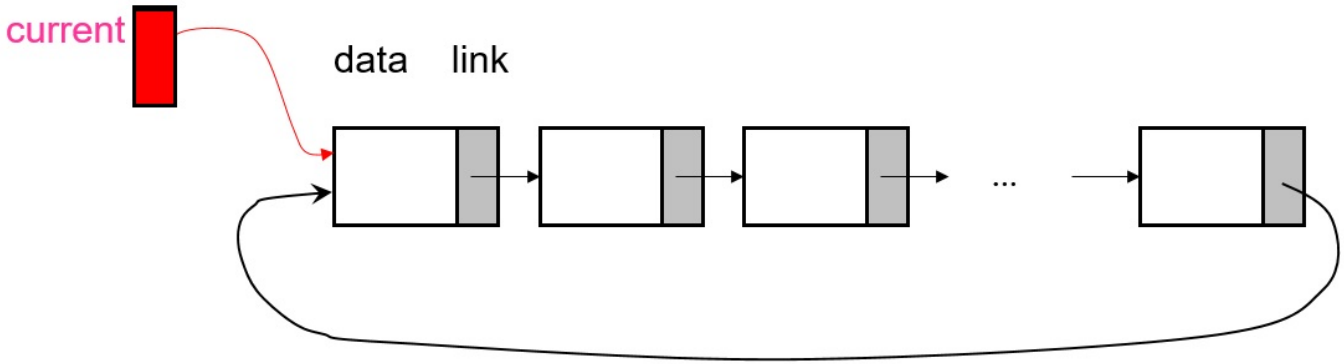


Hình: Doubly Linked List allows going forward and backward.



Hình: Insert an element in Doubly Linked List.

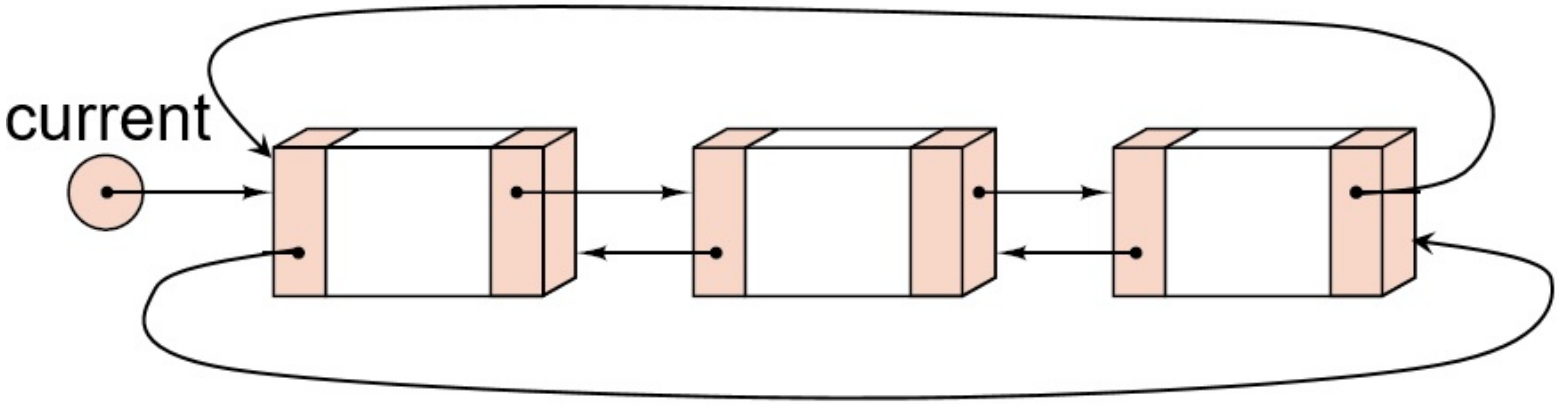
Circularly Linked List




```
node
   data <dataType>
   link <pointer>
end node
```

```
list
   current <pointer>
end list
```

Double circularly Linked List




```
node
   data <dataType>
   next <pointer>
   previous <pointer>
end node
```

```
list
   current <pointer>
end list
```

Comparison of implementations of list

Dr. Nguyen Ho Man Rang

Arrays: Pros and Cons

- **Pros**:
  - Access to an array element is fast since we can compute its location quickly.
- **Cons**:
  - If we want to insert or delete an element, we have to shift subsequent elements which slows our computation down.
  - We need a large enough block of memory to hold our array.

Linked Lists: Pros and Cons

- **Pros**:
  - Inserting and deleting data does not require us to move/shift subsequent data elements.
- **Cons**:
  - If we want to access a specific element, we need to traverse the list from the head of the list to find it which can take longer than an array access.

Comparison of implementations of list

- **Contiguous storage is generally preferable when**:
  - the entries are individually very small;
  - the size of the list is known when the program is written;
  - few insertions or deletions need to be made except at the end of the list; and
  - random access is important.

- **Linked storage proves superior when**:
  - the entries are large;
  - the size of the list is not known in advance; and
  - flexibility is needed in inserting, deleting, and rearranging the entries.